ASIC Flow Implementation on a 32 bit RISC V Processor using Cadence

Mohammed Abdul Raheem
Department of ECE, Muffakham Jah College of enegineering and Technology
Hyderabad, India.
abdulraheem@ieee.org

Mohammed Sabir Hussain
Department of ECE, Muffakham Jah College of Engineering and Technology
Hyderabad, India.
sabir.mj@gmail.com

Pathan Rehman Ahmed Khan
Department of ECE, Muffakham Jah College of Engineering and Technology
Hyderabad, India
prakprak2002@gmail.com

Abstract- **This document portrays the design & ASIC flow implementation of a single-cycle microarchitecture of RV32I. RISC-V is chosen for its open-source ISA, which makes it accessible without any cost and offers similar capabilities as proprietary architectures like ARM and x86. The RV32I microarchitecture was designed using Verilog and comprises modules such as Instruction Fetch, Register Memory, Instruction Memory, ALU, and Control Unit. The functionality of the design was verified in Cadence Simvision tool, and it was compiled and elaborated in Cadence NClaunch tool. The design was synthesized error-free in Cadence Genus tool using TSMC 45nm technology libraries. The RTL design was validated by performing a Logic Equivalence Check (LEC) in Cadence LEC-Conformal tool, which verified that the design is perfectly mapped to gate-level netlist. Physical design was implemented in Cadence Innovus tool, and the timing of the design was verified in Cadence Tempus tool using the constraint files provided by TSMC 45nm technology.**

Keywords- RV32I, RISC-V, ARM, ISA, RTL, Verilog, LEC, STA.

I. INTRODUCTION

RISC-V is open-source ISA, which means it can be modified and used without any restriction or licensing. It provides a greater ability for the engineers to develop the required embedded system at low cost and offering higher performance. RISC-V ISA consists of a. main set & a set of elective extension instructions. Base instruction set is common to all RISC-V implementations and contains instructions such as load/store instructions, arithmetic and logic instructions, and branch and jump instructions. The extensions such as Integer Multiplication and Division extension (M), Floating-Point extension (F), Cryptography extension (C) and many more, it provides flexibility to the designers to add extra functionality to the design. RV32I is the 1st of RISC-V ISA, and it includes main ISA

There are two main approaches to processor design: RISC & CISC. The features make RISC a better choice for embedded designers and enables it be a prominent processor in the field of "system on chip".

979-8-3503-7464-3/24/$31.00 ©2024 IEEE

RISC is a processor architecture that emphasizes simplicity and a small set of basic instructions. The RISC architecture was developed in the 1980s as an alternative to the traditional CISC architecture, which used a large set of complex instructions. The key idea behind RISC architecture is to make each instruction perform a very specific and simple task, so that the processor can execute instructions more quickly and efficiently. This is achieved by eliminating unnecessary instructions and focusing on the most frequently used instructions. Some of the key characteristics of a RISC processor include: 1. Large number of registers: RISC processors typically have many registers that can be accessed quickly, reducing the need to access main memory. 2. Simple instructions: RISC processors use a small set of simple instructions that are easy to decode and execute quickly. 3. Pipelining: RISC processors often use pipelining, a technique that allows multiple instructions to be executed simultaneously, increasing overall performance. 4. Load-store architecture: In a RISC processor, data can only be stored into registers from memory or stored from registers to memory using specific load and store

This work provides architectural design of a RISC-V base integer instruction set ISA, which of 32-bit address space and capable of executing R-type Instruction Format provided by RV32I ISA. The instructions such as " add, sub, xor, or, and, sll, srl, sra, slt and sltu" are executed by the design and each instruction was executed within a single clock cycle. The design is specified to operate at 500MHZ and it does function at the specified frequency without any violations.

II. RISC-V (RV32I) Instruction Set

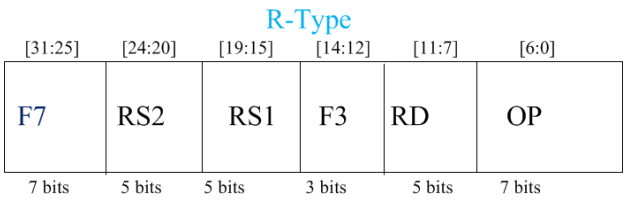
The RISC-V set is made up of four fundamental formats: R, I, S/B-,& U/J-type.

2024 IEEE International Conference of Electron Devices Society Kolkata Chapter (EDKCON) | 979-8-3503-7464-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/EDKCON62339.2024.10870604

These formats include a total of 47 instructions, which perform basic operations like data movement, arithmetic and logical, branching or jumping. Each instruction format has separate fields that are required for decoding the instruction and determining its type, such as R, I, S/B, U/J. The opcode field is used to determine the instruction format, while the function field specifies process performed by the instruction. Because the decoder logic for the four instruction formats is distinct, each format requires a different decoder logic.

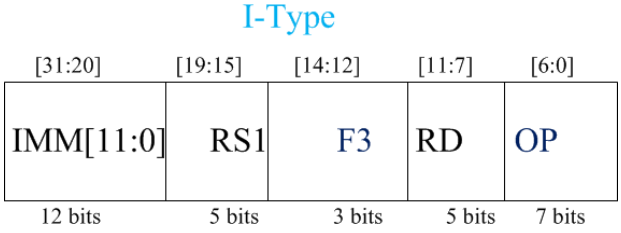
A. R- Type Commands

The R-type IF in RISC-V is referred to as the register type, and it consists of instructions that perform various operations on register data, such as addition, subtraction, bitwise operations like XOR, OR, AND, and shifting operations like SLL, SRL, SRA, SLT, and SLTU. The R-type instruction format is divided into six fields: F7, RS2, RS1, F3, RD, and OP. The function field is formed by combining F7 and F3, and it specifies the type of process to be done. The RS1 field specifies the location in memory of the first operand, which is a 5-bit register. The RS2 field indicates the place in memory of the second operand, which is also a 5-bit register. The OP field is a 7-bit field that specifies the opcode, which is different for different instruction formats. Finally, the RD field indicates the place in retention where the result of the ALU operation will be stored.



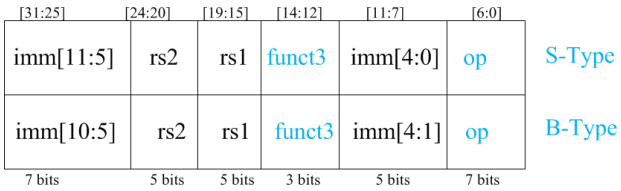
B. I-Type Commands

The I-type IF in RISC-V is known as Immediate type instructions. These instructions include immediate data, which is used as the second operand for ALU operations. The various instructions in the I-type format include ADDI, ORI, ANDI, SUBI, SLLI, SRLI, SRAI, SLTI, and SLTIU. The I-type instruction format is divided into 5 fields: IMM, RS1, F3, RD, and OP. The IMM field is a 12-bit field that specifies the value of the 2nd operand. Before performing the ALU procedure, this assess must be sign-stretched to 32 bits. The remaining fields are like those in the R-type format.



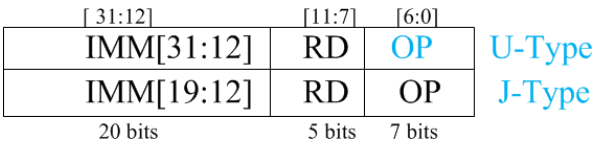
C. S/B-Type Commands

The S/B type IF in RISC-V is used for store & branch instructions. The S-type instructions perform stores of facts from a indicate to a memory setting, while the B-type instructions perform conditional branching based on the result of a previous operation. The S-type instruction format is divided into six fields: IMM[11:5], RS2, RS1, F3, IMM[4:0], and OP. The immediate field is split into two parts, IMM[11:5] & IMM[4:0], & they specify the offset from the base talk to of the memory spot where the data is to be stored. The RS1 field indicates the register memory spot that comprises the base address, while the RS2 field asserts the register retention place that holds the facts to be stored. The F3 field is a 3-bit field that indicates the operation to be implemented. The B-type instruction format is divided into six fields: IMM[10:5], RS2, RS1, F3, IMM[4:1], and OP. The immediate field is split into four parts, IMM [10:5], IMM[4:1], and they specify the offset from the current instruction address to the target address. The RS1 pitch requires the register memory position containing the first operand, while the RS2 field specifies the register memory location containing the second operand. The F3 field is a 3-bit field that identifies the process to be behaved, such as equal, not equal, less than, greater than, etc.



D. U/J-Type Commands

The U/J type IF in RISC-V is used for unconditional jumps and immediate loads. The U-type instruction format is used for immediate loads and is divided into two fields: IMM[31:12] and RD, OP. The immediate field specifies a 20-bit value i.e., sign- stretched to 32 bits & then added to the current instruction address to form the memory address for the load operation. RD field specifies the destination register where the loaded data is to be stored. The J-type instruction format is used for unconditional jumps and is divided into two fields: IMM[19:12], and OP. The immediate field specifies a 20-bit value i.e., sign- ranged to 21 bits and then inserted to the current instruction address to form the target discourse for the jump operation. The OP field specifies the opcode for the instruction. Both U & J type guidelines have 20-bit immediate fields, which allows for a wide range of instant values to be loaded or used for jump targets.



III. ARCHITECTURE DEVELOPMENT AND IMPLEMENTATION

RISC-V ISA which is of address space 32-bit is designed and the overview of the design implementation is described in the subsequent sections.

A. Top level Overview

The design of the processor can be divided into four main components: Instruction Fetch, Instruction Memory, Registers, and Control and ALU unit. The Control unit is responsible for processing the function and opcode bits of each instruction and generating control signals to determine which ALU operation is to be performed. It also generates control bits, such as the read/write bit, for accessing the register memory. The Instruction Memory stores the instructions in each memory location. The Program Counter (PC) is initially reset and increments by one location for each clock cycle. The instruction register is loaded with the instruction based on the value of the PC, and after decoding the instruction, the ALU unit performs various arithmetic and logical operations within a specific time frame and provides the output, which is written to the specified register.

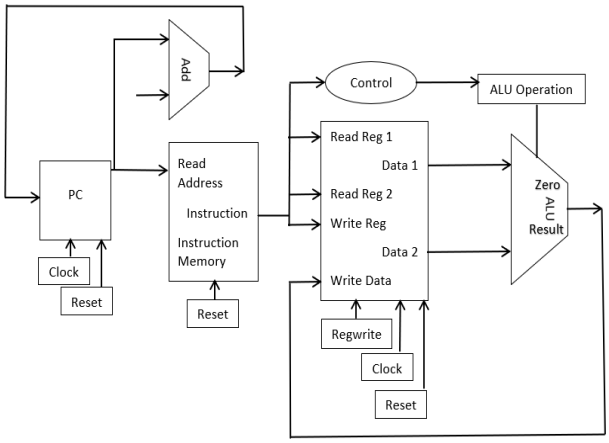


Fig 1: Block Diagram

B. Micro-Architecture

To implement the 32-bit RISC-V (RV32I) architecture, the microarchitecture is divided into two parts that work together: Datapath and Control unit. The datapath processes data in units called words and includes structures like memories, registers, and ALUs. As we are implementing the RV32I architecture, our datapath is 32 bits wide. The control unit receives instructions from the datapath and directs how they should be executed. It generates signals such as alu opcode, register enable, and memory write to control the operation of the datapath.

The hardware implementation can be divided into four stages:

1. Instruction Fetch

The PC supplies the memory position of the subsequent instruction to be achieved. With every +ve edge of the clock signal, the PC value is incremented by one location. Based on the current PC value, the instruction is fetched from the instruction memory & loaded keen on the instruction register.

2. Instruction Decode

Step:1 the processor decodes the instructions that are fetched from the Instruction register. The processor then reads the operands that are required for the register file, based on the instructions. The 32-bit instructions are divided into different fields, such as the opcode field, source-1 register address field, source-2 register address field, and the function field. The function field details the process to be executed by the ALU unit.

3. Execution

In this stage, all instructions are executed. This is where the ALU performs arithmetic and logical operations based on the register content received as the two inputs. The instructions specify the operation to be performed by the ALU and the destination address where the result should be stored back in the register memory.

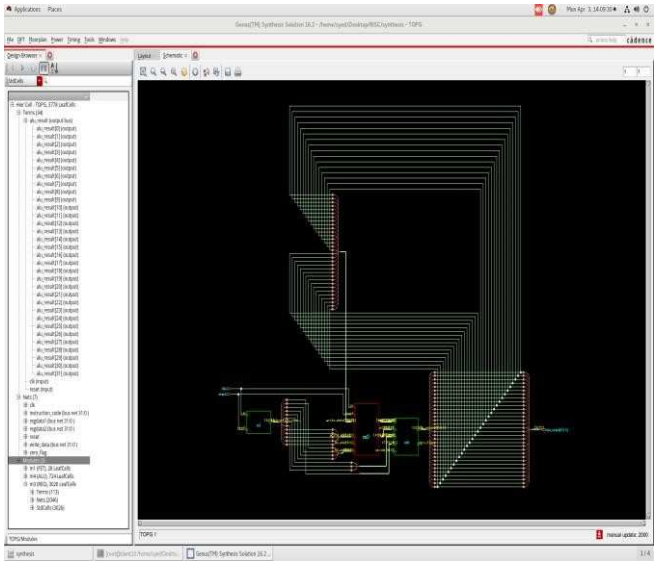


Fig 2: Synthesized Schematic

IV. RESULTS AND DISSCUSSION

The RTL design was successfully compiled in the Cadence NClaunch tool without any errors, and its functionality was verified in the Cadence Simvision tool.

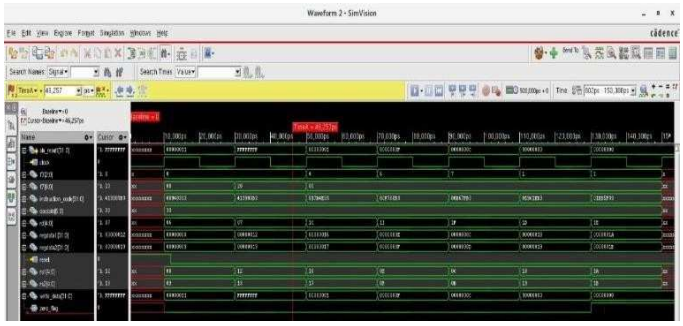


Fig 3: Simulation Result

The Cadence Genus tool was used to synthesize the design, utilizing the typical libraries of TSMC 45nm technology. The resulting reports for area, power, and quality of results (QOR) attached below.

.

```
Generated by:              Genus(TM) Synthesis Solution 16.23-s049_1
Generated on:              Mar 23 2023  11:49:02 am
Module:                    TOPG
Wireload mode:             enclosed
Area mode:                 timing library
============================================================

                      Leakage    Dynamic      Total
 Instance     Cells  Power(nW)  Power(nW)   Power(nW)
---------------------------------------------------------
TOPG           3462 225092.498 434175.134 659267.633
  m3           2619 201766.623 375261.406 577028.029
  m4            693  21056.258  39557.124  60613.382
    srl_23_23   145   3114.063  15380.487  18494.550
    sll_22_23   145   3101.289  18526.741  21628.030
    sub_18_23    67   2998.342   2042.648   5040.991
    add_17_23    34   2692.382   2847.767   5540.149
    lt_25_24     81   1184.954    745.137   1930.091
  m1            150   2269.617   4585.075   6854.692
    instr_mem1  140   1831.488   2712.922   4544.410
```

Fig 4: Power Report

```
============================================================
Generated by:            Genus(TM) Synthesis Solution 16.23-s049_1
Generated on:            Apr 01 2023  03:03:53 pm
Module:                  TOPG
Technology library:      slow
Operating conditions:    slow (balanced_tree)
Wireload mode:           enclosed
Area mode:               timing library
============================================================

 Instance          Module            Cells  Cell Area  Net Area   Total Area  Wireload
--------------------------------------------------------------------------------------
TOPG                                  2554      24317         0        24317    <none> (D)
  m3          REG                     1802      19202         0        19202    <none> (D)
  m4          ALU                      724       4979         0         4979    <none> (D)
    sll_22_23 shift_left_vlog_unsigned 162        861         0          861    <none> (D)
    srl_23_23 shift_right_vlog_unsigned 162       858         0          858    <none> (D)
    sub_18_23 sub_unsigned              67        683         0          683    <none> (D)
    add_17_23 add_unsigned_68           34        618         0          618    <none> (D)
    lt_25_24  lt_unsigned               81        347         0          347    <none> (D)
  m1          FET                       28        136         0          136    <none> (D)
    instr_mem1 INS_MEM                  18         61         0           61    <none> (D)

(D) = wireload is default in technology library
```

Fig 5: Area Report

```
Timing
--------

Clock  Period
-------------
clk    20000.0

  Cost     Critical           Violating
 Group    Path Slack   TNS      Paths
-------------------------------------
clk         18679.9    0.0          0
default    No paths    0.0
-------------------------------------
Total                  0.0          0

Instance Count
--------------
Leaf Instance Count                2554
Physical Instance count               0
Sequential Instance Count          1059
Combinational Instance Count       1495
Hierarchical Instance Count           9

Area
----
Cell Area                             24316.926
Physical Cell Area                    0.000
Total Cell Area (Cell+Physical)       24316.926
Net Area                              0.000
Total Area (Cell+Physical+Net)        24316.926

Max Fanout                            64 (m3/n_112)
Min Fanout                            0 (zero_flag)
Average Fanout                        2.7
Terms to net ratio                    4.1207
Terms to instance ratio               4.2240
Runtime                               36.668654000000004 seconds
Elapsed Runtime                       44 seconds
Genus peak memory usage               610.97
```

Fig 6: QOR Report

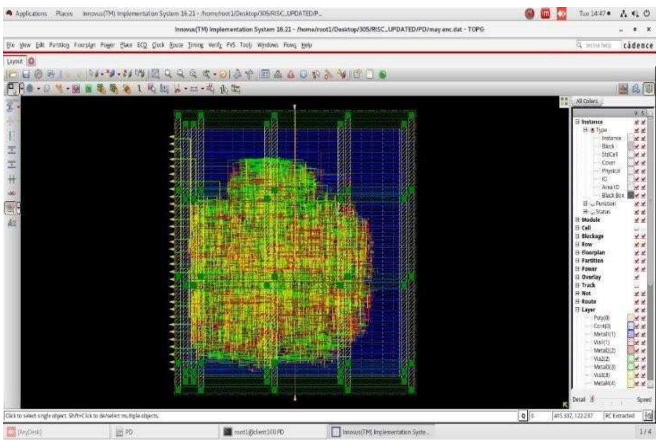


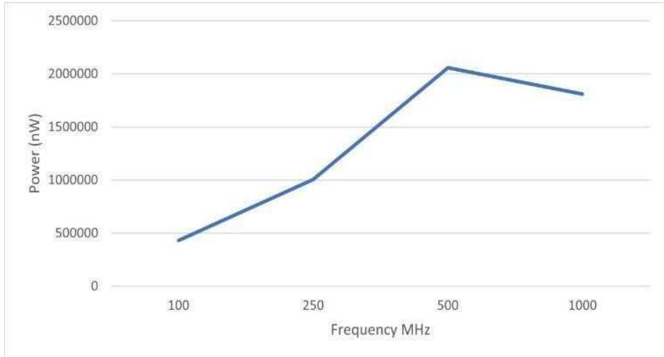
Fig 7: Final Layout View

Netlist generated in the synthesis process is loaded into Cadence Innovus along with design constraint files such as .sdc(synopsis design constraints) file, .lef(Library exchange format) file. Entire flow of PD (physical design) was implemented in Cadence Innovus tool and the above figure give final layout view of the design.

Power Vs Frequency

Frequency	Leakage Power(nw)	Dynamic Power(nw)	Total power (nw)
1. 100	226.076	430056.651	1809147.634
2. 250	219.422	1003870.382	2057394.673
3. 500	240.32	2057154.353	1004089.804
4. 1000	214.716	1808932.918	430282.727

Table 1: Power Vs Frequency



Graph 1: Comparison between Dynamic Power and Frequency

The above graph depicts that the dynamic power increases linearly as frequency increases.

V. CONCLUSION

In this work, we projected RISC-V 32-bit single cycle microarchitecture that could perform various operations specified in the instructions of RV32I instruction formats. Verilog is used to model the hardware and verified the functionality of the design on Cadence Simvision tool. The design is synthesized using typical libraries of TSMC 45nm technology in Cadence Genus tool and analyzed the reports generated after synthesis. Cadence Innovus tool is used for Foor planning, Power planning, Placement, CTS, Routing and PVS. We analyzed the reports of physical design and verified that the design is meeting all its requirements and finally generated GDS-II file**.**

ACKNOWLEDGEMENT

I acknowledge and thanks to Dr. Ayesha Naaz Head of ECE Dept at MJCET facilitated us to carry-out this work at research centre of MJCET. Also I thank the Alumin Mr. M A Khadeer , Mr Mohd Waqar and Priyanka continuous carried out the RISC Processor work and contributed in the work.

REFERENCES

[1] K. Asanovi and D. A. Patterson, “Instruction sets should be free: The case for risc-v,” EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, Aug 2014.

[2] A. Waterman, Y. Lee, D. A. Patterson, K. Asanovic, V. I. U. level Isa, A. Waterman, Y. Lee, and D. Patterson, “The risc-v instruction set manual,” 2014.

[3] Aneesh Raveendran, Vinayak Baramu Patil, David Selvakumar, Vivian Desalphine, “A RISC-V Instruction Set Processor-Micro architecture Design and Analysis”, 2016 International Conference on VLSI Systems, Architectures, Technology and Applications (VLSI-SATA).

[4] "Computer Organization and Design- the hardware/software interface", 3rd edition by David A. Patterson and John L. Hennessy, pp. 370-412.

[5] Raj Kumar Singh Parihar, Shivananda Reddy, “Design of 16-BitRISC Processor”, Birla Institute of Technology and Science, May 2006.

[6] M. A. Raheem, H. Gupta, K. Fatima and O. Adil, "A high-speed reversible low-power Error Tolerant Adder," 2012 Asia Pacific Conference on Postgraduate Research in Microelectronics and Electronics, Hyderabad, India, 2012, pp. 178-183, doi: 10.1109/ PrimeAsia.2012.645864